

Scientific and Rendering Density Resource Separation

Stage 11E0a: disjoint ownership of numerical-field and browser/mesh admission

mdstats

2026-07-25

Contents

1 Purpose and ownership	1
2 Scientific resource domain	1
3 Rendering resource domain	2
4 Runtime compatibility resolution	2
5 Immutability, serialization, and signatures	2
6 Failure behavior	3
7 Acceptance requirements	3
8 Method provenance	3

1 Purpose and ownership

Stage 11E0a separates resources needed to construct a scientific density field from resources needed to render or serialize that field.

Scientific owner:

```
mdstats.analysis.density.resources.ScientificDensityResourcePolicy
```

Rendering owner:

```
mdstats.plotting.density_resource_policy.DensityRenderingResourcePolicy
```

The two policies are intentionally non-substitutable.

2 Scientific resource domain

`ScientificDensityResourcePolicy` contains only limits that can alter whether a scientific field can be evaluated or stored:

- field count;
- dense logical voxel count;
- source/quadrature sample count;
- nonzero logical nodes;
- stored block values and block count;
- kernel-pair work;
- planning workspace;

- realization workspace;
- CIC contribution work;
- primary memory;
- numerical threads; and
- numerical wall time.

It contains no:

- mesh-cell or mesh-face limit;
- visual target face count;
- Plotly trace count;
- WebGL profile;
- browser payload size;
- cloud rendering point count; or
- HTML serialization budget.

Scientific limits may reject field construction before allocation. They must not be inferred from a particular browser or rendering mode.

3 Rendering resource domain

`DensityRenderingResourcePolicy` contains:

- a browser mesh budget;
- a mesh-face contract; and
- a cloud-rendering point limit.

It contains no field-count, voxel, sample, kernel-pair, or numerical-work limit. It is accepted only by plotting/rendering code. The analysis density facade does not accept it.

4 Runtime compatibility resolution

Until revision-44 GR1/GR5 move the numerical resolution and resource planners, the scientific policy may be resolved through the established runtime resource module. The resulting public policy contains only scientific fields and records the compatibility resolver in metadata.

Explicit limits may tighten runtime-derived limits but cannot relax the active memory, thread, or wall-time allocation.

The compatibility translation into the existing numerical producers is:

```
max_fields
max_total_voxels
max_samples
max_nonzero_nodes
max_stored_block_values
max_blocks
max_kernel_pairs
max_planning_bytes
max_workspace_bytes
max_cic_contributions
```

No rendering argument is generated.

5 Immutability, serialization, and signatures

Scientific policy metadata are recursively immutable and JSON-compatible. The policy signature covers all scientific limits, primary runtime limits, schema, and metadata.

Rendering policy serialization labels its domain as `density_rendering` and states that scientific limits are absent. Scientific serialization labels its domain as `scientific_density` and contains no rendering fields.

6 Failure behavior

The scientific policy fails closed for:

- nonpositive or nonintegral count limits;
- non-finite/nonpositive wall time;
- planning or workspace bytes exceeding primary memory;
- unsupported schema;
- non-JSON metadata; or
- combining an explicit policy with separate resolution overrides.

The rendering policy fails closed for invalid browser or mesh contracts and a nonpositive cloud-point limit.

7 Acceptance requirements

- Scientific and rendering policy JSON payloads have disjoint domains.
- The scientific facade accepts only `ScientificDensityResourcePolicy`.
- A rendering policy cannot be passed as a scientific policy.
- Scientific field construction does not invoke browser admission.
- Browser/mesh limits cannot change scientific field values.
- Existing low-level runtime resolution remains regression-compatible.

8 Method provenance

This stage introduces no borrowed numerical method. The separation of resource domains and their non-substitutability are project-specific ownership rules.